Driver Proc Implementation

Wednesday, June 10, 2026 7:56 PM

Proc → debugging

↳ System behavior

Output we need to create file under Proc

ptoc is a file system

#include "module.h" ↳ Data structure

#include proc_fs.h ①

The Driver that we will implement doesn't depend on a device

Implementation will be based on the given functionality in proc_fs.h

How we will initialize the Driver ?

① insmod ——— .ko

② init function

Deinitialized

① rmmod podcast

② De init function

```c
/** Init function
 * @brief used in static in tree and out of tree [insmod]
 */
__init static int my_module_init_function(void)
{
    printk("hello from init function \n");
    pdir = proc_create("podcast", 0666, NULL, &pops);
    return 0;
}
```

Starting

E_OK

module_init(==)

why static ?

eliminate other Drivers to initialize Driver

Why Linux make the init function in init section ?

RAM Limited

↳ Small RAM

Remove init section from the RAM

Why --exit Section?

Static → --exit → will not be part of your Memory

Dynamic → rmmod

MODULE_LICENCE
MODULE_AUTHOR → in order your Drier to compile

proc_file_operations

```
struct proc_ops {
    unsigned int proc_flags;
    int (*proc_open)(struct inode *, struct file *);
    ssize_t (*proc_read)(struct file *, char __user *, size_t, loff_t *);
    ssize_t (*proc_read_iter)(struct kiocb *, struct iov_iter *);
    ssize_t (*proc_write)(struct file *, const char __user *, size_t, loff_t *);
    /* mandatory unless nonseekable_open() or equivalent is used */
    loff_t  (*proc_lseek)(struct file *, loff_t, int);
    int (*proc_release)(struct inode *, struct file *);
    __poll_t (*proc_poll)(struct file *, struct poll_table_struct *);
    long    (*proc_ioctl)(struct file *, unsigned int, unsigned long);
#ifdef CONFIG_COMPAT
    long    (*proc_compat_ioctl)(struct file *, unsigned int, unsigned long);
#endif
    int (*proc_mmap)(struct file *, struct vm_area_struct *);
    unsigned long (*proc_get_unmapped_area)(struct file *, unsigned long, unsigned long, unsigned long, unsigned long);
} __randomize_layout;
```

open fd
read
write

file ← operations
↳ Map

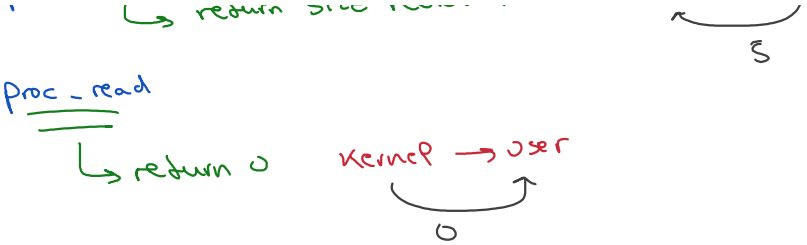
```
proc_create(name, mode, parent, proc_ops)
```

file name | rw | NULL | data structure

Linux C
↳ Doesn't use std c library
In order to debug we need to printk
↳ dmesg

proc - write ... ... recieved

User → Kernel


proc_read
return 0
kernel → user
0
5


echo hello > podcast
5
ssize_t podcast_write(struct file *fi, const char __user *user, size_t size, loff_t * off)
{
// int ret = copy_from_user(arr, user, 4);
// printk("Podcast is write %d \n", arr[0]);
printk("Podcast is write \n");
return size;
}


offset
5
0 1 2 3 4 5
off = 0